

Automatic Differentiation to Improve Parameter Estimation for Partially Observed Markov Processes

K. Tan¹, A. J. Abkemeier², G. Hooker¹ and E. L. Ionides²

¹Department of Statistics, University of Pennsylvania

²Department of Statistics, University of Michigan

June 2, 2026

Abstract

Automatic differentiation (AD) has driven recent advances in machine learning, including deep neural networks and Hamiltonian Markov Chain Monte Carlo methods. Partially observed nonlinear stochastic dynamical systems have proved resistant to AD techniques because widely used particle filter algorithms yield an estimated likelihood function that is discontinuous as a function of the model parameters. To resolve this, we create a theoretical framework that embeds two existing AD particle filter methods within a new class of algorithms. This new class permits a bias-variance tradeoff and hence a mean squared error substantially lower than the existing algorithms. This allows us to develop likelihood maximization algorithms suited to the Monte Carlo properties of the AD gradient estimate. Our algorithms require only a differentiable simulator for the latent dynamic system; by contrast, most previous approaches to AD likelihood maximization for particle filters require access to the system's transition probabilities. Numerical results indicate that a hybrid algorithm that uses AD to refine a coarse solution from an iterated filtering algorithm shows substantial improvement on current state-of-the-art methods on a challenging scientific benchmark problem.

1 Introduction

Many scientific models involve highly nonlinear stochastic dynamic systems possessing significant random variation in both the process dynamics and the measurements. Commonly, the latent system is modeled as a Markov process, giving rise to a partially observed Markov process (POMP) model, also known as a hidden Markov models or a state space model. POMP models arise in fields as diverse as optimal control (Singh et al., 2023), epidemiology (He et al., 2010; Stocks et al., 2018), ecology (Knappe and de Valpine, 2012) and finance (Kim and Stoffer, 2008; Bretó, 2014). Despite their ubiquity, estimation and inference for this broad class of models remains a challenging problem. We develop full-information plug-and-play inference for POMP models that takes advantage of automatic differentiation (AD) computation while controlling the Monte Carlo bias and variance of the resulting estimator. These desiderata have not previously been combined, and we discuss each component in turn.

Full-information methods are those which make statistically efficient use of data, typically via likelihood-based or Bayesian methods that avoid dependence on summary statistics. The particle

filter, also known as sequential Monte Carlo, serves as the foundation for various full-information inference algorithms for POMP models, since it provides an unbiased estimate of the likelihood function (Del Moral, 2004), enabling Bayesian inference via particle Markov chain Monte Carlo (Andrieu et al., 2010) and likelihood-based inference via iterated filtering algorithms IF1 (Ionides et al., 2006) and IF2 (Ionides et al., 2015).

A POMP model inference algorithm is said to be plug-and-play if it does not need to evaluate the transition density of the latent Markov process, enabling an arbitrary model simulator to be plugged into the algorithm, a convenient property that facilitates scientific discovery (He et al., 2010). Some plug-and-play methods depend on the construction of low-dimensional summary statistics (Wood, 2010; Toni et al., 2009), sacrificing statistical information for computational convenience. Here, our mission is to extend current computational capability among methods which maintain statistical efficiency and the plug-and-play property. Plug-and-play methods are sometimes called likelihood-free (Owen et al., 2015), and it may be counter-intuitive that likelihood-based inference is possible using likelihood-free methods. Basic particle filters, also known as bootstrap filters, possess the plug-and-play property. Iterated particle filtering algorithms, such as IF1 and IF2, have shown that plug-and-play likelihood maximization is practical in various scientific investigations (King et al., 2008; Blake et al., 2014; Pons-Salort and Grassly, 2018; Subramanian et al., 2021; Fox et al., 2022; Drake et al., 2023). However, iterated filtering algorithms have slow convergence near the MLE (Doucet et al., 2013).

Recent advances in AD for gradient estimation using particle filters (Naesseth et al., 2018; Jonschkowski et al., 2018; Corenflos et al., 2021; Ścibior and Wood, 2021; Singh et al., 2023) have drawn attention to AD as a tool for inference in POMP models. Widespread adoption of these approaches have been limited by their bias (Naesseth et al., 2018; Jonschkowski et al., 2018), high variance (Poyiadjis et al., 2011; Ścibior and Wood, 2021), computational expense quadratic in the number of particles (Corenflos et al., 2021; Chen and Li, 2024), or by the loss of the plug-and-play property (Poyiadjis et al., 2011; Ścibior and Wood, 2021; Singh et al., 2023; Chen and Li, 2024). We present a new algorithm which (for reasons which will be discussed later) we call a *Differentiated Measurement Off-Parameter* (DMOP) particle filter. DMOP corresponds to the derivative of a smooth variant of the particle filter that we call MOP, though DMOP is conveniently calculated using AD without the need to actually implement the full MOP filter. MOP has the special property that, unlike most particle filters, the gradient of the filtered log-likelihood estimate provides a consistent estimate of the gradient of log-likelihood. Particle filters have previously been adapted to provide a smooth estimate of the likelihood function (Svensson et al., 2018; Malik and Pitt, 2011), but MOP and DMOP differ from these by possessing the plug-and-play property.

DMOP trades bias against variance via a discount factor, $\alpha \in [0, 1]$. We show that DMOP- α possesses a bias-variance tradeoff that permits both low bias and low variance for suitable choices of α strictly in $(0, 1)$, both in theory (Theorem 5) and practice (Figure 1). We show that DMOP- α coincides with the biased gradient estimator of Naesseth et al. (2018) when $\alpha = 0$. In a limit as the number of particles increases, DMOP- α matches the high-variance gradient estimate from (Poyiadjis et al., 2011; Ścibior and Wood, 2021) when $\alpha = 1$ (Theorem 1). At $\alpha = 1$, the variance can become prohibitively high, as discovered by these researchers and others (Arya et al., 2022). However, the variance rapidly reduces as α decreases, and so a choice of α close to one can provide a low-bias, low-variance, plug-and-play gradient estimate with computational scaling linear in the number of particles.

In practice, likelihood optimization via gradient descent with DMOP- α converges quickly within

a neighborhood of the maximum but struggles to reach this neighborhood in the presence of local minima and saddle points. By contrast, iterated filtering algorithms provide a relatively fast and stable way to identify this neighborhood. We therefore build a simple but effective hybrid algorithm called *Iterated Filtering with Automatic Differentiation* (IFAD) that warm-starts first-order or second-order gradient methods with a preliminary solution obtained from iterated filtering. We implement our algorithm using JAX (Bradbury et al., 2018), which also enables us to take advantage of graphical processing unit (GPU) hardware. Our methodology is included in `pypomp` (Abkemeier et al., 2025), an open-source software package for POMP modeling leveraging GPU acceleration and automatic differentiation.

Promising numerical results indicate that IFAD beats IF2 (and by the numerical results of Ionides et al. (2015), also IF1 and the Liu-West filter (Liu and West, 2001)) on a challenging problem in epidemiology, the Dhaka cholera model of King et al. (2008). Empirically, the benchmark tests demonstrate that IFAD is the most significant methodological advance since Ionides et al. (2015) for plug-and-play statistical inference on POMP models. The choice of models and methods for benchmarking is justified in Section S8.

We also demonstrate empirical advances using DMOP- α for Bayesian inference. In Section 6, we use the DMOP- α gradient estimates within a No-U-Turn Sampler (NUTS) (Homan and Gelman, 2014) in conjunction with an empirical Bayes-type prior estimated with IF2 to reduce the burn-in period of particle MCMC (Andrieu et al., 2010) on the Dhaka cholera model of King et al. (2008) from the 700,000 iterations in Fasiolo et al. (2016) to just 500. To the best of our knowledge, attaining rapid mixing on this challenging problem has not previously been achieved.

2 Problem Setup

Consider an unobserved Markov process $\{X(t), t \geq t_0\}$, with discrete-time observations $Y_1, \dots, Y_N$ realized at values $y_1^*, \dots, y_N^*$ with times $t_1, \dots, t_N$. The process is parameterized by an unknown parameter $\theta \in \Theta \subseteq \mathbb{R}^p$, where the state $X(t)$ takes values in the state space $\mathcal{X} \subseteq \mathbb{R}^d$, the observations Y_n take values in $\mathcal{Y}$, and we write $X_n = X(t_n)$. We suppose that the discrete-time latent process model has a density $f_{X_n|X_{n-1}}(x_n | x_{n-1}; \theta)$. The existence of this density is necessary to define the likelihood function, but plug-and-play algorithms do not explicitly evaluate it. Instead, they have access to a corresponding simulator, which write as `processn` ($x_n | x_{n-1}; \theta$). We set $f_{Y_n|X_n}(y_n | x_n; \theta)$ to be the measurement density, which can be directly evaluated. We call $f_{X_{1:n}|Y_{1:n}}(x_{1:n} | y_{1:n}^*; \theta)$ the posterior distribution of states, and $f_{X_n|Y_{1:n}}(x_n | y_{1:n}^*; \theta)$ the filtering distribution at time t_n . All joint and conditional densities are defined on a probability space $(\Omega, \Sigma, \mathbb{P})$ which is also assumed to enable construction of independent replicates and all random variables defined in our algorithms. For the algorithmic interpretation of our theory, we identify the random number seed with an element of the sample space $\omega \in \Omega$. The random seed determines the sequence of pseudo-random numbers generated by a computer, just as $\omega \in \Omega$ generates the sequence of random variables.

3 Off-Parameter Particle Filters

The *Measurement Off-Parameter with discount factor α* (MOP- α), particle filter is defined by the pseudocode in Algorithm 1. Note that this algorithm has the plug-and-play property. We suppose that the simulator is a differentiable function of θ for every fixed ω . This condition is equivalent to

the *reparameterization trick* (Corenflos et al., 2021); by writing the generated random numbers as parameter-dependent transformation from a reference distribution—e.g., the quantile transform of a uniform random variable—we can differentiate their values, and hence the simulation, with respect to parameters. The discrete resampling step of the bootstrap particle filter has been the primary point of concern for the use of AD to estimate derivatives for particle filters. The discontinuity of this resampling (as a function of θ , for fixed ω) means that the derivative of the expectation is not consistently estimated by the expectation of the derivative. MOP- α is a mathematical construction designed to avoid this discontinuity. We begin with a filter that resamples particles with weights calculated at a baseline parameter value, ϕ , and then corrects these weights to target the filtering distribution at another parameter value, θ , yielding the name *measurement off-parameter*. The name is chosen by analogy with similar *off-policy* algorithms used for reinforcement learning in which a policy that was not implemented is evaluated by reweighting the rewards of the policy that was actually applied. If we fix ω and ϕ , the sample indices do not change with θ , but the re-weighting scheme does, allowing MOP- α to be differentiated with respect to θ . When $\theta = \phi$, we have $w_{n,j}^{P,\theta} = w_{n-1,j}^{F,\theta} = 1$ and MOP- α reduces to the standard particle filter; this is the setting we use in practice, but the construction allows us to define a derivative. One can think of MOP- α as reconstructing local smooth approximations of the discontinuous likelihood estimates produced by a particle filter for a fixed number of particles J and fixed seed ω . These approximate the likelihood at θ in a neighborhood around ϕ , with the intent that differentiating these smooth log-likelihood reconstructions yields a score estimate.

DMOP- α (Algorithm 2) computes the fixed-seed derivative of the likelihood estimate obtained by MOP- α at $\theta = \phi$. To better understand the implementation DMOP- α we review some background for AD. Standard AD consists of two evaluation passes: a forward pass in which the function is evaluated and a graph built of the evaluation process, followed by a backward pass where the derivative is evaluated by recursive use of the chain rule on the computation graph (Griewank and Walther, 2008). The two evaluation passes in MOP- α at ϕ and θ correspond directly to the two evaluation passes in DMOP- α . Differentiation of MOP- α with respect to θ requires a preliminary evaluation at ϕ and subsequent differentiation with respect to θ for the second pass which we obtain through AD. In practice, however, we only work in the $\theta = \phi$ setting in which case MOP- α corresponds to the standard particle filter and we only need to make one pass. In order to apply AD in this setting without using two passes, we must both instantiate the weighting scheme (even though the weights are always 1) and distinguish which components of the weights correspond to which pass. This can be achieved using the *stop-gradient* capability that appears in Algorithm 2 where its application in the weight correction step indicates that AD should apply the chain rule to the numerator but not the denominator of the correction; note that it is this construction that yields weights which are always 1, but which also have non-zero derivatives.

MOP- α has similarities with Svensson et al. (2018), in which particles from a previous run at ϕ are reweighted to evaluate the likelihood at any sufficiently nearby parameter θ for 0-th order optimization. However, their method is not plug-and-play because they use transition densities to reweight particles, and the variance of their log-likelihood estimate has an exponential dependence in the horizon, N , when $\theta \neq \phi$. Our approach bypasses these issues – the first since the reparameterization trick allows us to consider particle paths dependent on θ , and the second as using AD allows us to only evaluate the likelihood and score estimates when $\theta = \phi$.

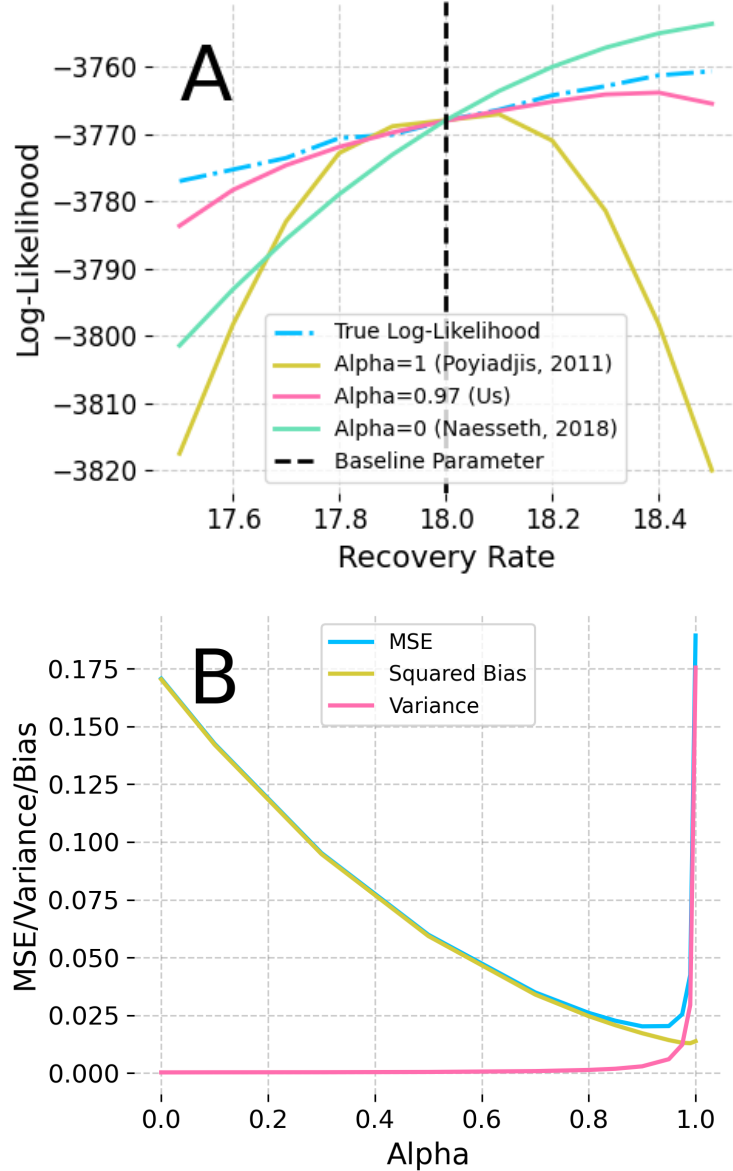


Figure 1: **A:** Non-differentiable likelihood estimates and smooth off-parameter approximations that hold locally around the baseline parameter ϕ . **B:** Illustration of the bias-variance tradeoff induced by the discounting hyperparameter α . We display the MSE of score estimates for the trend in transmission at the MLE for the Dhaka cholera model of King et al. (2008). Our novel contribution of $\alpha \in (0, 1)$ significantly improves the quality of the likelihood approximation and the score estimates.

Algorithm 1 MOP- α

Input: Number of particles J , timesteps N , measurement model $f_{Y_n|X_n}(y_n^*|x_n, \theta)$, simulator $\text{process}_n(\cdot|x_n; \theta)$, evaluation parameter θ , baseline parameter ϕ , seed ω .

First pass: Set $\theta = \phi$ and fix ω , yielding $X_{n,j}^{P,\phi}$, $X_{n,j}^{F,\phi}$, $g_{n,j}^\phi$.

Second pass: Fix ω , and filter at $\theta \neq \phi$:

Initialize particles $X_{0,j}^{F,\theta} \sim f_{X_0}(\cdot; \theta)$, weights $w_{0,j}^{F,\theta} = 1$.

For $n = 1, \dots, N$:

Accumulate discounted weights, $w_{n,j}^{P,\theta} = (w_{n-1,j}^{F,\theta})^\alpha$.

Simulate process model, $X_{n,j}^{P,\theta} \sim \text{process}_n(\cdot | X_{n-1,j}^{F,\theta}; \theta)$.

Measurement density, $g_{n,j}^\theta = f_{Y_n|X_n}(y_n^* | X_{n,j}^{P,\theta}; \theta)$.

Compute $L_n^{B,\theta,\alpha} = \sum_{j=1}^J g_{n,j}^\theta w_{n,j}^{P,\theta} / \sum_{j=1}^J w_{n,j}^{P,\theta}$.

Conditional likelihood under ϕ , $L_n^\phi = \frac{1}{J} \sum_{m=1}^J g_{n,m}^\phi$.

Select resampling indices $k_{1:J}$ with $\mathbb{P}(k_j = m) \propto g_{n,m}^\phi$.

Obtain resampled particles $X_{n,j}^{F,\theta} = X_{n,k_j}^{P,\theta}$.

Calculate corrected weights $w_{n,j}^{F,\theta} = w_{n,k_j}^{P,\theta} g_{n,k_j}^\theta / g_{n,k_j}^\phi$.

Compute $L_n^{A,\theta,\alpha} = L_n^\phi \cdot \sum_{j=1}^J w_{n,j}^{F,\theta} / \sum_{j=1}^J w_{n,j}^{P,\theta}$.

Return: likelihood estimate $\hat{\mathcal{L}}^A(\theta) = \prod_{n=1}^N L_n^{A,\theta,\alpha}$ or $\hat{\mathcal{L}}^B(\theta) = \prod_{n=1}^N L_n^{B,\theta,\alpha}$, and filtering distributions $\{(X_{n,j}^{F,\theta}, w_{n,j}^{F,\theta})\}_{n,j=1}^{N,J}$.

Algorithm 2 DMOP- α

Input: Number of particles J , timesteps N , measurement model $f_{Y_n|X_n}(y_n^*|x_n, \theta)$, simulator $\text{process}_n(\cdot|x_n; \theta)$, parameter θ .

Forward pass: AD constructs a computation graph for the likelihood estimate, $\hat{\mathcal{L}}^A(\theta) = \prod_{n=1}^N L_n^{A,\theta,\alpha}$ or $\hat{\mathcal{L}}^B(\theta) = \prod_{n=1}^N L_n^{B,\theta,\alpha}$.

Backward pass: AD evaluates the gradient estimate, $\hat{\mathcal{D}}^A(\theta)$ or $\hat{\mathcal{D}}^B(\theta)$.

Initialize particles $X_{0,j}^{F,\theta} \sim f_{X_0}(\cdot; \theta)$, weights $w_{0,j}^{F,\theta} = 1$.

For $n = 1, \dots, N$:

Accumulate discounted weights, $w_{n,j}^{P,\theta} = (w_{n-1,j}^{F,\theta})^\alpha$.

Simulate process model, $X_{n,j}^{P,\theta} \sim \text{process}_n(\cdot | X_{n-1,j}^{F,\theta}; \theta)$.

Measurement density, $g_{n,j}^\theta = f_{Y_n|X_n}(y_n^* | X_{n,j}^{P,\theta}; \theta)$.

Compute $L_n^{B,\theta,\alpha} = \sum_{j=1}^J g_{n,j}^\theta w_{n,j}^{P,\theta} / \sum_{j=1}^J w_{n,j}^{P,\theta}$.

Select resampling indices $k_{1:J}$ with $\mathbb{P}(k_j = m) \propto g_{n,m}^\theta$.

Obtain resampled particles $X_{n,j}^{F,\theta} = X_{n,k_j}^{P,\theta}$.

Calculate corrected weights $w_{n,j}^{F,\theta} = w_{n,k_j}^{P,\theta} g_{n,k_j}^\theta / \text{stop_gradient}(g_{n,k_j}^\theta)$.

Compute $L_n^{A,\theta,\alpha} = L_n^\theta \cdot \sum_{j=1}^J w_{n,j}^{F,\theta} / \sum_{j=1}^J w_{n,j}^{P,\theta}$.

Return: likelihood estimate $\hat{\mathcal{L}}^A(\theta) = \prod_{n=1}^N L_n^{A,\theta,\alpha}$ or $\hat{\mathcal{L}}^B(\theta) = \prod_{n=1}^N L_n^{B,\theta,\alpha}$, and gradient estimate $\hat{\mathcal{D}}^A(\theta)$ or $\hat{\mathcal{D}}^B(\theta)$.

3.1 Some algorithmic details

The resampling indices, which we write as $k_j \sim \text{Categorical}(g_{n,1}^\phi, \dots, g_{n,J}^\phi)$, are a function of ϕ for any value of θ . For the categorical distribution, we use systematic resampling (Arulampalam et al., 2002; King et al., 2016) which usually outperforms multinomial resampling.

In Algorithm 1, the conditional likelihoods are reweighted by a correction factor $w_{n,j}^{P,\theta}$ that accumulates over filtering steps to account for the resampling under ϕ . Writing $g_{n,j}^\theta = f_{Y_n|X_n}(y_n^*|X_{n,j}^{P,\theta}; \theta)$ for the measurement density and $L_n^\phi = \frac{1}{J} \sum_{m=1}^J g_{n,m}^\phi$ for the conditional likelihood estimate under ϕ , we obtain two estimates of the conditional likelihood under θ ,

$$L_n^{B,\theta,\alpha} = \frac{\sum_{j=1}^J g_{n,j}^\theta w_{n,j}^{P,\theta}}{\sum_{j=1}^J w_{n,j}^{P,\theta}}, \quad L_n^{A,\theta,\alpha} = L_n^\phi \cdot \frac{\sum_{j=1}^J w_{n,j}^{F,\theta}}{\sum_{j=1}^J w_{n,j}^{P,\theta}}, \quad (1)$$

where we recursively correct the weights for each particle for the cumulative error incurred by the off-parameter resampling:

$$w_{n,j}^{P,\theta} = (w_{n-1,j}^{F,\theta})^\alpha, \quad w_{n,j}^{F,\theta} = w_{n,k_j}^{P,\theta} g_{n,k_j}^\theta / g_{n,k_j}^\phi, \quad w_{0,j}^{F,\theta} = 1. \quad (2)$$

The before-resampling estimate $L_n^{B,\theta,\alpha}$ is preferable in practice, as it has slightly lower variance than the after-resampling estimate $L_n^{A,\theta,\alpha}$, but the latter is useful in deriving properties of the MOP- α gradient estimate such as that in Theorem 1.

3.2 Discounted Weights and a Bias-Variance Tradeoff

Heuristically, α controls the exponentially decaying memory of MOP and DMOP. Importantly, α is irrelevant for MOP at $\theta = \phi$ when the weights are always 1 and therefore also irrelevant for the function evaluation in the first pass of DMOP, but crucially not in the second when the gradients are evaluated. If $\alpha = 1$, the gradient of the weights, $w_{n,j}^{P,\theta}$ and $w_{n,j}^{F,\theta}$, accumulates as n increases. This leads to numerical instability and high variance unless N is small. For $\alpha < 1$, the weights from previous timesteps are discounted as in Equation (2), allowing for lower variance at the expense of bias in the gradient estimate.

When $\alpha = 1$, MOP-1 maintains complete memory of each particle’s ancestral trajectory. Theorem 1 shows that DMOP-1 recovers the consistent but high-variance score estimate of Poyiadjis et al. (2011). When $\alpha = 0$, MOP-0 considers only single-step transition dynamics, and DMOP-0 recovers the low-variance but asymptotically biased score estimate of Naesseth et al. (2018). Ścibior and Wood (2021) showed that this matches the gradient of a vanilla particle filter when resampling terms are dropped.

Theorem 1 (DMOP-0 and DMOP-1 Functional Forms). *Writing $\nabla_\theta \hat{\ell}^\alpha(\theta)$ for the gradient estimate yielded by DMOP- α and using the after-resampling conditional likelihood estimate so that $\hat{\mathcal{L}}(\theta) = \prod_{n=1}^N L_n^{A,\theta,\alpha}$, when $\alpha = 0$,*

$$\nabla_\theta \hat{\ell}^0(\theta) = \frac{1}{J} \sum_{n=1}^N \sum_{j=1}^J \nabla_\theta \log f_{Y_n|X_n}(y_n^*|x_{n,j}^{F,\theta}; \theta),$$

yielding Naesseth et al. (2018)’s estimator with the bootstrap filter. When $\alpha = 1$,

$$\nabla_\theta \hat{\ell}^1(\theta) = \frac{1}{J} \sum_{j=1}^J \nabla_\theta \log f_{Y_{1:N}|X_{1:N}}(y_{1:N}^*|x_{1:n,j}^{A,F,\theta}),$$

yielding the estimator of Poyiadjis et al. (2011) for the bootstrap filter.

We defer the proof to Appendix ?? . The argument relies on a useful decomposition of the after-resampling estimate $L_n^{A,\theta,\alpha}$, repeated applications of the log-derivative identity that $\nabla_x \log(f(x)) = (\nabla_x f(x))/f(x)$, and noting $\theta = \phi$ implies that $w_{n,j}^{P,\theta}$ evaluates to 1.

The result in Theorem 1 provides mathematical intuition for why Ścibior and Wood (2021)’s use of the stop-gradient procedure in the particle filter recovers the estimate of Poyiadjis et al. (2011). When $\theta = \phi$, the correction $g_{n,k_j}^\theta/g_{n,k_j}^\phi$ is equivalent to $g_{n,k_j}^\theta/\text{stop_gradient}(g_{n,k_j}^\theta)$, and is implemented in this way in practice. As such, the use of the stop-gradient in Ścibior and Wood (2021) can be seen as the MOP- α weight correction for $\alpha = 1$, and $\theta = \phi$.

4 From Gradients to Maximum Likelihood Estimation

To design an effective procedure for likelihood maximization using DMOP- α , we carry out iterated filtering then automatic differentiation (IFAD). A basic IFAD approach is presented in Algorithm ?? . This requires a Hessian estimate $H(\theta)$ with minimum eigenvalue no smaller than c , but we allow for the possibility of a trivial estimate, $H = I_p$, in which case the AD search becomes stochastic gradient descent (SGD). Algorithm ?? is convenient for theoretical analysis, but two further modifications are helpful in practice. Our best numerical results were obtained by putting the gradient estimate into the Adam optimizer (Kingma and Ba, 2015) together with learning rate decay. Numerical results are provided in Section 5.

Algorithm 3 IFAD

Input: Number of particles J , timesteps N , IF2 and MOP- α cooling schedules η_m , MOP- α discounting parameter α , θ_0 , $m = 0$.

Run initial IF2 search under cooling schedule η_m to obtain $\{\Theta_j, j = 1, \dots, J\}$, set $\theta_m := \frac{1}{J} \sum_{j=1}^J \Theta_j$.

While procedure not converged:

 Run Algorithm 1 to obtain $\hat{\ell}(\theta_m)$.

 Set $g(\theta_m) := \nabla_\theta(-\hat{\ell}(\theta_m))$, and consider any $H(\theta_m)$ s.t. $\lambda_{\min}(H(\theta_m)) \geq c$.

 Update $\theta_{m+1} := \theta_m - \eta_m(H(\theta_m))^{-1}g(\theta_m)$, $m := m + 1$.

Return $\hat{\theta} := \theta_m$.

The convergence of IF2 (and so the first stage of IFAD) to a neighborhood of the MLE happens rapidly in practice. However, even when applying much larger amounts of Monte Carlo effort, IF2 fails to get within seven **ADJUST FOR BEST IF2 AND NEW UNDERSTANDING OF MLE** log-likelihood units of the maximum on this problem (Figure 3). For this example, IF2, without assistance from AD, struggles to reach the inferentially critical region within one log-likelihood unit of the maximum, even when additional computationally intensive techniques are used, such as repeated pruning and profiling,

The AD stage confers a linear convergence rate for IFAD, up to a small region near the MLE shrinking in J , under the usual smoothness and strong convexity assumptions in convex optimization theory, as we show below in Theorem 2.

Theorem 2 (Linear Convergence of IFAD). *Assume strongly convex and smooth $-\ell$, $\gamma I \preceq \nabla_\theta^2(-\ell) \preceq \Gamma I$, and assumptions (A1–A5) in Section 7. Choose the learning rate η so $\eta \leq \frac{c(1-\beta)}{2\Gamma}$. Consider the second stage of IFAD (Algorithm 3), stopping if $\|\nabla_\theta \hat{\ell}^\alpha(\theta_m)\| \leq (1 + \sigma)\epsilon$, where $\sigma \geq \frac{4\Gamma}{(1-\beta)}$ for some $\beta \in (0, 1)$. Choose the learning rate η so $\eta \leq \frac{c(1-\beta)}{2\Gamma}$. Then, for α sufficiently*

close to 1 and J sufficiently large (depending on ϵ and δ), we have that $\|\nabla_{\theta}\hat{\ell}^{\alpha}(\theta_m) - \nabla_{\theta}\ell^{\alpha}(\theta_m)\| \leq \epsilon$ and $H(\theta_m) \succ cI_p \succ 0$ for all m , and that

$$\ell(\theta^*) - \ell(\theta_{m+1}) \leq \left(1 - \eta\beta \frac{8\gamma}{9c}\right) (\ell(\theta^*) - \ell(\theta_m)) \text{ with probability at least } 1 - \delta.$$

The proof of Theorem 2 uses results from randomized numerical linear algebra, building on Theorem 6 of Roosta-Khorasani and Mahoney (2016). Details are deferred to Appendix ???. This result shows that particle estimates for the gradient can enjoy the same linear convergence rate on well-conditioned problems as gradient descent with access to the score.

We therefore see that the second stage of IFAD converges linearly to the MLE if (1) the log-likelihood surface is well-conditioned in a neighborhood of the MLE and (2) the first stage of IFAD successfully reaches a (high-probability) basin of attraction of the MLE. Within a neighborhood of the MLE that grows as more data become available (Ning et al., 2021), local quadratic approximations such as local asymptotic normality apply under general conditions (Le Cam and Yang, 2000). This suggests that the linear convergence may occur often in practice.

5 Application to a Cholera Transmission Model

The cholera transmission model that King et al. (2008) developed for Dhaka, Bangladesh, has been used to benchmark the performance of various POMP inference methods (Ionides et al., 2015; Fasiolo et al., 2016; Wycoff et al., 2026), and we employ it here for the same purpose. This model categorizes individuals in a population as susceptible, $S(t)$, infected, $I(t)$, and recovered, $R(t)$ and so is called an SIRS compartmental model. In this case, the compartment $R(t)$ is further subdivided into three recovered compartments $R^1(t)$, $R^2(t)$, $R^3(t)$ denoting varying degrees of cholera immunity. We write $P(t)$ for the total population, and M_n for the cholera deaths in each month. As in King et al. (2008), the transition dynamics follow a stochastic differential equation (SDE):

$$\begin{aligned} dS &= (k\epsilon R^k + \delta(P - S) - \lambda(t)S) dt + dP - (\sigma SI/P) dB, \\ dI &= (\lambda(t)S - (m + \delta + \gamma)I) dt + (\sigma SI/P) dB, \\ dR^1 &= (\gamma I - (k\epsilon + \delta)R^1) dt, \quad \dots \\ dR^k &= (k\epsilon R^{k-1} - (k\epsilon + \delta)R^k) dt, \end{aligned}$$

with Brownian motion $B(t)$, cholera death rate m , recovery rate γ , mean immunity duration $1/\epsilon$, standard deviation of the force of infection σ , and population death rate $\delta = 0.02$. The force of infection, λ_t , is modeled by splines $(s_j)_{j=1}^6$

$$\lambda_t = \exp\left(\beta_{\text{trend}}(t - t_0) + \sum_{j=1}^6 \beta_j s_j(t)\right) \frac{I}{P} + \exp\left(\sum_{j=1}^6 \omega_j s_j(t)\right),$$

where the coefficients $(\beta_j)_{j=1}^6$ model seasonality in the force of infection, β_{trend} models the trend in the force of infection, and the ω_j represent seasonality of a non-human environmental reservoir of disease. The measurement model for observed monthly cholera deaths is $Y_n \sim \mathcal{N}(M_n, \tau^2 M_n^2)$, where $M_n = \gamma \int_{t_{n-1}}^{t_n} I(s) ds$ is the modeled cholera deaths in that month.

We numerically solve this SDE using Euler's method with a time step of 1/20 months, matching (King et al., 2008). This example is a partially observed hypo-elliptical SDE, and methodology

specific to this model class has been previously studied (Ditlevsen and Samson, 2019). Our method applies to a class of latent Markov processes larger than Gaussian SDEs, permitting the use of Lévy noise (Bhadra et al., 2011) or other discontinuities.

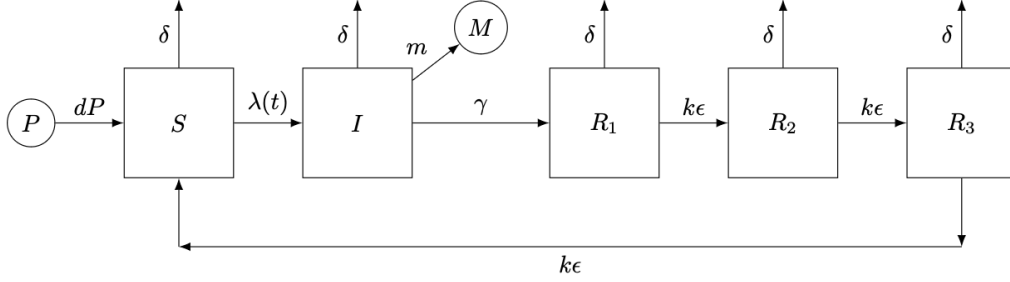


Figure 2: A compartment flow diagram for the King et al. (2008) Dhaka cholera model.

5.1 Results

We test IFAD with $\alpha = 0, 0.97, 1$ against IF2 on a global search problem for the Dhaka cholera model using our implementation in `pypomp` 0.4.6.0 (Abkemeier et al., 2025). For each method, we perform 100 searches, initialized with 100 starting parameter vectors drawn uniformly from the same wide bounding box used in Ionides et al. (2015). To ensure a fair comparison under comparable computational effort, we run each method for about as long as it takes for IFAD-0.97 to converge, as it converges to the highest log-likelihood in the shortest time. For each method, we choose algorithmic parameters that maximize the log-likelihood best for that method while finishing within the allotted time. We run standard IF2 for 650 iterations, while IFAD-0.97 is run using a short IF2 warm-start of 175 iterations followed by the Adam optimizer for 175 iterations. IFAD-0 and IFAD-1 are run for 60 IF2 iterations and 225 Adam iterations. To assess whether other methods can catch up to IFAD-0.97 given more time, we separately run the IF2-only approach for 4 times as many iterations and the IFAD variants with 4 times as many Adam iterations, slowing the cooling rate for each as well. A summary of these optimization settings is provided in Section ??, and the complete source code to reproduce the numerical results is available at <https://github.com/pypomp/dmop>.

We summarize our findings in Table 1 and Figures 3, 4, and 5. Because estimating the log-likelihoods at every iteration of IF2 with particle filtering for Figure 5 lengthens the computation time and thus compromises the fair comparison between methods, we obtain the times in Table 1 and Figure 5 by running the methods again without the extra calls to the particle filter. Every other value displayed for the Dhaka results comes from the runs with the extra particle filter calls.

Examining Table 1, IFAD-0.97 can attain a log-likelihood of -3744.17, which surpasses IF2 even when IF2 runs for 4 times as long. IFAD-1 comes close in the time it takes for IFAD-0.97 to hit its maximum, but it needs to be run for an extended time to reach that maximum and partly close the gap in median log-likelihood. IFAD-0 falls close to 4 log-units short of IFAD-0.97 even with extended time, ostensibly due to having high bias. Based on Figure 5, IFAD-0 and IF2-only plateau heavily after being run for long enough and ostensibly would not catch up to IFAD-0.97 without excessive time and effort.

Previously, an MLE at a log-likelihood of -3748.6 was reported by King et al. (2016). This MLE was obtained with much computational effort, using many global and local IF2 searches and

	Best Log-Likelihood	Median Log-Likelihood	IF2 time (s)	DMOP time (s)
<i>Comparable computational effort</i>				
IFAD-0.97	-3744.17	-3745.77	244	615
IFAD-1	-3745.12	-3749.12	89	788
IF2	-3747.82	-3755.41	889	-
IFAD-0	-3748.69	-3753.88	90	788
<i>Extended computational effort</i>				
IFAD-0.97	-3744.19	-3744.22	244	2459
IFAD-1	-3744.19	-3745.46	89	3152
IF2	-3747.02	-3750.14	3557	-
IFAD-0	-3747.97	-3752.41	90	3152

Table 1: Maximum log-likelihood found by IF2 and IFAD under both comparable and extended computational effort. IFAD-0.97 performs the best among all methods.

with the assistance of likelihood profiling, 8 years after the model was first proposed by King et al. (2008). Meanwhile, Ionides et al. (2015) only achieve a maximum log-likelihood of -3768.6 , and King et al. (2008) only -3793.4 . Using the improved computational efficiency of `pypomp` (elaborated upon in Section 8), IF2 alone can easily surpass all of these previous results within a wall clock time of 15 minutes, and IFAD-0.97 holds the record at -3744.17 within a slightly shorter time. [It wouldn't actually take that long to run the IF2 searches in R, so I don't think that really explains why -3747.82 hadn't been easily hit previously.](#)

Notably, the log-likelihood and parameter estimates for IFAD-0.97 frequently exhibit much less Monte Carlo variance than that of the other methods, as can be seen in Figures 3 and 4. IFAD in general exhibits less Monte Carlo variance than IF2, especially for the parameter estimates. IFAD-0 has the second-lowest Monte Carlo variance for most parameter estimates, although it has the lowest for the initial state proportions S_0 and $R_{1,0}$. That said, these are also the most biased estimates, and IFAD-1 tends to agree more closely with IFAD-0.97. Because multiple searches are employed in large part to mitigate the effects of Monte Carlo variance, the lower Monte Carlo variance of IFAD-0.97 suggests that fewer searches are needed when running IFAD-0.97 to obtain a log-likelihood close to the maximum. Lowering the Monte Carlo variance also narrows the width of Monte-Carlo-adjusted profile confidence intervals (Ionides et al., 2017).

5.2 IFAD Outperforms Both IF2 and DMOP Alone

While IF2 quickly approaches a neighborhood of the MLE (Figure 5), IF2 alone ultimately fails to achieve the last few log-likelihood units, as IF2 struggles to get much closer than about 3 log-likelihood units of the MLE even when run for an extended time (Table 1). Conversely, we have encountered many failed searches with gradient descent alone. This is a difficult, nonconvex, and noisy problem with 23 estimated parameters, and gradient descent without a warm-start gets stuck in local minima and saddle points.

IFAD, in comparison, approaches the MLE quickly due to the IF2 warm-start (Figure 5) and also succeeds at refining the coarse solution found by the warm-start with DMOP gradient steps to find the MLE. IFAD therefore successfully combines the best qualities of IF2 and DMOP, out-

performing either of them alone. Monte Carlo replication is appropriate for IFAD, or any Monte Carlo algorithm used to solve a challenging numerical problem, but Figure 5 shows that IFAD can find higher likelihood values more quickly and more reliably than the previous state-of-the-art. Since replicated searches are inevitably needed for Monte Carlo maximization of this challenging likelihood maximization task, it is important to give extra consideration to the right tail of the distributions in Figure 3 when comparing the capabilities of the methods.

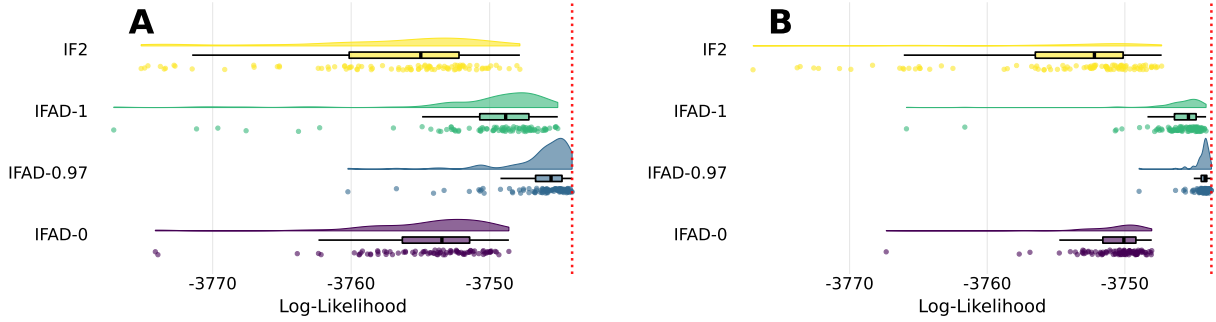


Figure 3: Raincloud plot of the searches. (a) Comparable computational effort. (b) Extended computational effort. Log-likelihoods lower than -3800 are omitted. IFAD-0.97 outperforms all other methods. The dotted red line shows the maximized log-likelihood.

6 Application to Bayesian Inference

Particle MCMC (Andrieu et al., 2010) is arguably the most popular method for full-information plug-and-play Bayesian inference for POMP models. However, particle Metropolis-Hastings often mixes slowly, requiring long burn-in periods. For instance, Fasiolo et al. (2016) reported needing 700000 burn-in iterations for effective sampling in the King et al. (2008) model, even after coarsening each Euler timestep to reduce the computational burden. To alleviate this, we employ a No U-Turn Sampler (NUTS) (Homan and Gelman, 2014) powered by DMOP- α , with a nonparametric empirical prior constructed from density estimation of the parameter swarm from the IF2 warm-start. Using NUTS with MOP- α and this prior significantly lowers the mixing time of the Markov chain to as little as 500 iterations (Figure 6). NUTS has been implemented with previous differentiable particle filters (Rosato et al., 2022) but not in our plug-and-play context with control over the bias/variance tradeoff.

Without the IF2 prior, NUTS with the MOP- α estimate alone tended to diverge. In contrast, particle Metropolis-Hastings, with or without said prior, fails to effectively explore the parameter space. We display these results in Figures ?? and ?? in Appendix ?. This speedup in the mixing time of particle MCMC by over three orders of magnitude provides hope that particle Bayesian inference might be increasingly employed for the complex scientific models commonly encountered in areas like epidemiology.

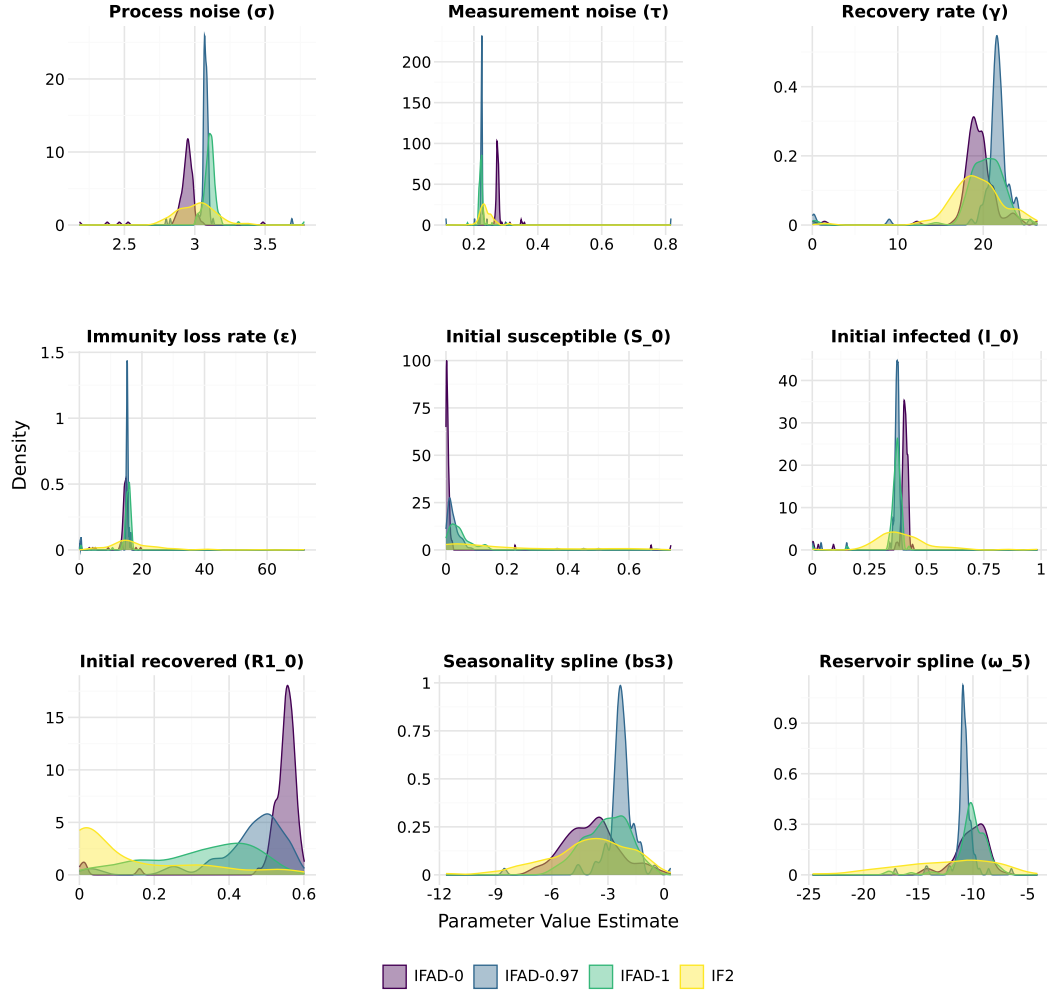


Figure 4: Density plots of the final parameter estimates across the 100 replicates for nine key parameters of the cholera model under extended computational effort.

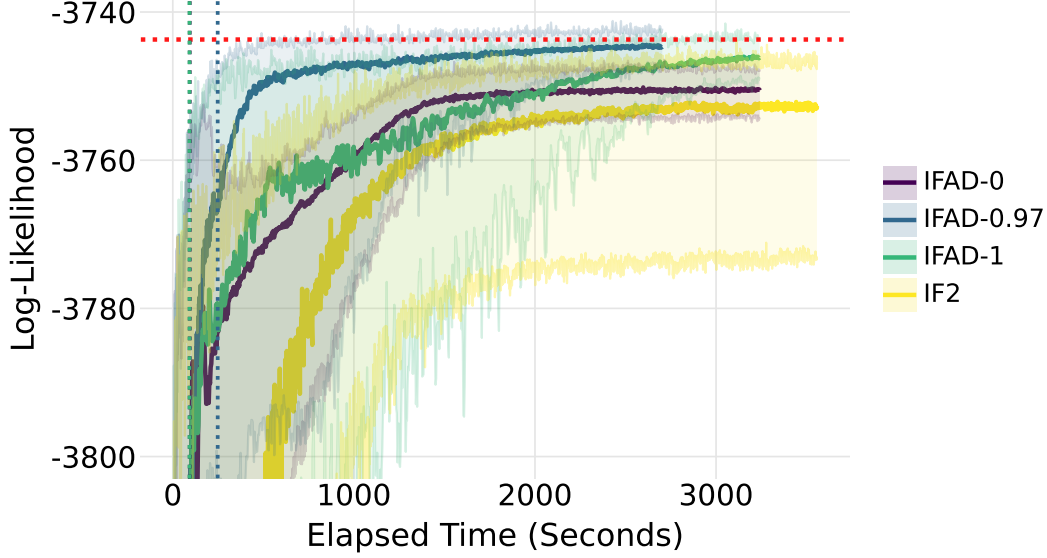


Figure 5: Optimization progress of IFAD and IF2 under extended computational effort. The shaded regions cover the area between the 100th and 10th percentiles of log-likelihood estimates at each iteration. We use a dotted red line to display the MLE. The vertical dotted lines indicate where the IFAD runs switch from IF2 to Adam optimization. Some log-likelihood estimates appear to be above the MLE; this is because the estimates prior to the final iteration use just 1 replicate instead of 36 replicates to estimate the log-likelihood, which leads to larger Monte Carlo variance.

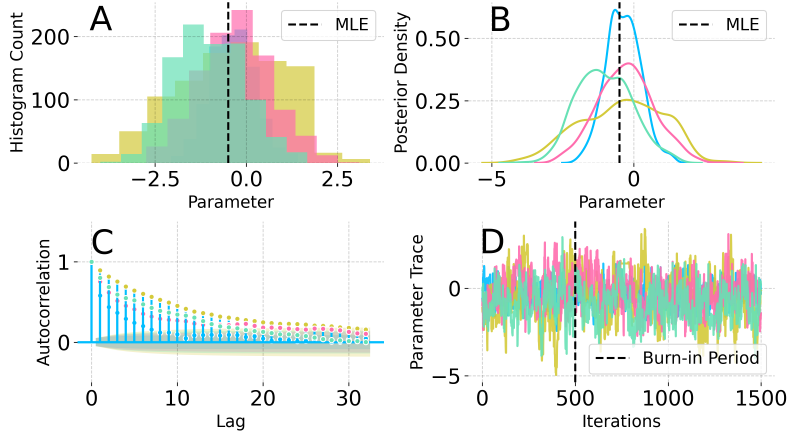


Figure 6: Convergence diagnostics for NUTS with 4 chains, where we plot 1000 samples from the posterior of β_{trend} within the Dhaka cholera model of King et al. (2008). **A:** Histogram of posterior samples. **B:** density estimate of the posterior. **C:** Autocorrelation plot for MCMC samples. **D:** Trace plot for MCMC samples, pre and post burn-in. NUTS mixes quickly when regularized by an empirical prior with a standard deviation of 6.3% of the MLE, and the posterior estimates from each chain share roughly the same posterior mode.

7 Theoretical Analysis of MOP- α

Here, we will show that MOP- α targets the filtering distribution and likelihood under θ , is consistent when $\alpha = 1$, and characterize rates for its bias and variance under different choices of α . To do so, we require the following assumptions:

- (A1) **Continuity of the Likelihood.** $\ell(\theta)$ has more than two continuous derivatives in a neighborhood $\{\theta : \ell(\theta) > \lambda_1\}$ for some $\lambda_1 < \sup_{\varphi} \ell(\varphi)$.
- (A2) **Bounded Process Model.** There exist $\underline{M}, \bar{M}$ such that $0 < \underline{M} \leq f_{X_n|X_{n-1}}(x_n|x_{n-1}; \theta) \leq \bar{M} < \infty$.
- (A3) **Bounded Measurement Model.** There exist $\underline{G}, \bar{G}$ such that $0 < \underline{G} \leq f_{Y_n|X_n}(y_n^* | x_n; \theta) \leq \bar{G} < \infty$, and $G'(\theta)$ with $\|\nabla_{\theta} \log f_{Y_n|X_n}(y_n^* | x_n; \theta)\|_{\infty} \leq G'(\theta) < \infty$.
- (A4) **Bounded Gradient Estimates.** There are functions $G(\theta), H(\theta) : \Theta \rightarrow [0, \infty)$ uniformly bounded by $G^*, H^* < \infty$, so the MOP- α gradient and Hessian estimates at $\theta = \phi$ are almost surely bounded by $G(\theta)$ and $H(\theta)$ for all α .
- (A5) **Differentiable Density Ratios and Simulator.** The measurement density, $f_{Y_n|X_n}(y_n^*|x_n; \theta)$, and simulator have more than two continuous derivatives in θ .

In particular, assumptions (A2) and (A3) imply that the POMP model satisfies a strong mixing condition, and as such are fairly strong but commonplace in the literature (Karjalainen et al., 2023; Del Moral, 2004). The fact that the likelihood estimate yielded by MOP- α has more than two continuous derivatives in θ follows from the construction of the likelihood estimate in equations 1 and 2, as well as Assumptions (A3) and (A5).

7.1 MOP- α Targets the Filtering Distribution

We show here that MOP- α targets the filtering distribution under θ and is strongly consistent for the likelihood under θ when $\alpha = 1$ or $\theta = \phi$. Employing $\alpha < 1$ lead to inconsistency when θ deviates from ϕ , but this bias disappears as θ approaches ϕ .

While the result is presented here as specific to MOP- α , we prove a more general result in Appendix ???. This is a strong law of large numbers for triangular arrays of particles resampled arbitrarily, with resampling probabilities not proportional to the targeted distribution of interest, but reweighted to correct for the cumulative discrepancy between the resampling and the target distribution.

Theorem 3 (MOP- α Targets the Filtering Distribution and Likelihood). *When $\alpha = 1$ or $\theta = \phi$, MOP- α targets the filtering distribution under θ and is strongly consistent for the likelihood under θ . That is, for $\pi_n(\theta) = f_{X_n|Y_{1:n}}(x_n|y_{1:n}^*; \theta)$ and any measurable and bounded functional h and for $\hat{\mathcal{L}}(\theta) = \prod_{n=1}^N L_n^{A,\theta,\alpha}$ or $\prod_{n=1}^N L_n^{B,\theta,\alpha}$, it holds that*

$$\frac{\sum_{j=1}^J h(x_{n,j}^{F,\theta}) w_{n,j}^{F,\theta}}{\sum_{j=1}^J w_{n,j}^{F,\theta}} \xrightarrow{a.s.} E_{\pi_n(\theta)}[h(X_n)], \quad \hat{\mathcal{L}}(\theta) \xrightarrow{a.s.} \mathcal{L}(\theta).$$

Proof. We provide a proof sketch here, deferring the details to Appendix ???. When $\theta = \phi$, the ratio $g_{n,j}^{\theta}/g_{n,j}^{\phi} = 1$ regardless of α , reducing to the vanilla particle filter. When $\alpha = 1$ and $\theta \neq \phi$, suppose inductively that $\{(X_{n-1,j}^{F,\theta}, w_{n-1,j}^{F,\theta})\}_{j=1}^J$ targets $f_{X_{n-1}|Y_{1:n-1}}(x_{n-1}|y_{1:n-1}^*; \theta)$. It can then

be shown that $\{(X_{n,j}^{P,\theta}, w_{n,j}^{P,\theta})\}_{j=1}^J$ targets $f_{X_n|Y_{1:n-1}}(x_n|y_{1:n-1}^*; \theta)$, that $\{(X_{n,j}^{P,\theta}, w_{n,j}^{P,\theta} g_{n,j}^\theta)\}_{j=1}^J$ targets $f_{X_n|Y_{1:n}}(x_n|y_{1:n}^*; \theta)$, and that weighting the particles by $(X_{n,j}^{F,\theta}, w_{n,j}^{F,\theta}) = (X_{n,k_j}^{P,\theta}, w_{n,k_j}^{P,\theta} g_{n,k_j}^\theta / g_{n,k_j}^\phi)$, re-sampling the k_j with probabilities proportional to g_{n,k_j}^ϕ , also targets $f_{X_n|Y_{1:n}}(x_n|y_{1:n}^*; \theta)$. Estimating the likelihood with the before-resampling estimator $\hat{\mathcal{L}}(\theta) = \prod_{n=1}^N L_n^{B,\theta,\alpha}$, we see that the strong consistency is a direct consequence of our result that $\{(X_{n,j}^{P,\theta}, w_{n,j}^{P,\theta})\}$ targets $f_{X_n|Y_{1:n-1}}(x_n|y_{1:n-1}^*; \theta)$. $\square$

7.2 DMOP-1 Score Estimate Is Consistent

We take advantage of the equivalence between DMOP- α and the derivative of MOP- α to investigate the former by studying the latter. Although we have shown the DMOP-1 gradient estimate yields the estimate of Poyiadjis et al. (2011) and Ścibior and Wood (2021) when applied to the bootstrap filter, those authors estimate the Fisher score by $\frac{1}{J} \sum_{j=1}^J \nabla_\theta \log f_{X_{0:N}, Y_{1:N}}(x_{0:n,j}^{A,F,\theta}, y_{1:N}^*; \theta)$ and not $\frac{1}{J} \sum_{j=1}^J \nabla_\theta \log f_{Y_{1:N}|X_{1:N}}(y_{1:N}^* | x_{1:n,j}^{A,F,\theta}; \theta)$. It is therefore not immediately apparent that these two converge to the same thing. As such we directly show the consistency of the MOP-1 gradient estimate below.

Theorem 4 (Consistency of MOP-1 Gradient Estimate). *The gradient estimate of MOP- α when $\alpha = 1$, $\theta = \phi$ is strongly consistent for the score: $\nabla_\theta \hat{\ell}_J^1(\theta) \xrightarrow{a.s.} \nabla_\theta \ell(\theta)$ as $J \rightarrow \infty$.*

We present a proof outline here, postponing the full argument to Appendix ???. Assumption (A4) lets us show uniform equicontinuity of the sequence for almost every $\omega \in \Omega$, allowing us to apply Arzela-Ascoli and Theorem 3 to establish uniform convergence for the derivatives as a sequence $(\nabla_\theta \hat{\mathcal{L}}_J^1(\theta)(\omega))_{J \in \mathbb{N}}$. This is a sufficient condition for swapping the limit and derivative to show that for almost every $\omega \in \Omega$, $\lim_{J \rightarrow \infty} \nabla_\theta \hat{\mathcal{L}}_J^1(\theta)(\omega) = \nabla_\theta \lim_{J \rightarrow \infty} \hat{\mathcal{L}}_J^1(\theta)(\omega) = \nabla_\theta \mathcal{L}(\theta)$.

We now provide a result showing that DMOP- α has a desirable bias-variance tradeoff when $0 < \alpha < 1$. This is achieved because it combines favorable properties from the low-variance but asymptotically biased estimate of Naesseth et al. (2018) and the high-variance but consistent estimate of Poyiadjis et al. (2011). As the bias itself is difficult to analyze, we instead analyze the MSE and variance. The below result applies for any $\alpha \in [0, 1)$, but not $\alpha = 1$, as the gradient estimate when $\alpha = 1$ has no forgetting properties.

Theorem 5 (MSE and Variance of DMOP- α). *When $\alpha \in (0, 1)$ and $\theta = \phi$, define $\psi_k(\alpha) = (\alpha^k + \alpha^{k+1} - \alpha)/(1 - \alpha)$. There exists an $\epsilon > 0$ depending on $\bar{M}, \underline{M}, \bar{G}, \underline{G}$ as in Karjalainen et al. (2023) such that the MSE and variance of DMOP- α are:*

$$\mathbb{E} \|\nabla_\theta \ell(\theta) - \nabla_\theta \hat{\ell}^\alpha(\theta)\|_2^2 \lesssim \min_{k \leq N} N p G'(\theta)^2 \left(k^2 J^{-1} + (1 - \epsilon)^{\lfloor k/(c \log(J)) \rfloor} + k + \psi_k(\alpha) \right), \quad (3)$$

$$\text{Var}(\nabla_\theta \hat{\ell}^\alpha(\theta)) \lesssim \min_{k \leq N} N p G'(\theta)^2 \left(\frac{k^2}{(1 - \alpha)^2 J} + \frac{\alpha^k}{1 - \alpha} N \right). \quad (4)$$

We provide an outline, deferring the full proof to Appendix ???. The variance bound can be reduced to the approximation error between the score estimate from DMOP- α and a variant called DMOP- (α, k) where the discounted weights are truncated at lag k . The discount factor, α , ensures strong mixing. The covariance of the derivative of MOP- (α, k) are controlled with Davydov's inequality and a standard L^p error bound for the particle filter. The MSE bound considers the error between the target derivative and DMOP- $(1, k)$, and the error between DMOP- $(1, k)$ and DMOP- α . The latter can be shown to be $\tilde{O}(N p G'(\theta)^2 (k + \psi_k(\alpha)))$, and the former can be controlled with a

result on the the forgetting of the particle filter from Karjalainen et al. (2023) and the very same L^p error bound mentioned above.

Theorem 5 provides theoretical understanding of the empirically favorable bias-variance trade-off enjoyed by DMOP- α in Figure 1. The first term in the MSE bound in Equation 3 corresponds to particle approximation error, the second mixing error, and the third and fourth the error between the DMOP- $(1, k)$ and DMOP- α score estimates. As α goes to 1, choosing k appropriately, the first, third and fourth term (corresponding to the variance) increases, while the second term (corresponding to the bias) decreases. Likewise, the first term in the variance bound in Equation 4 corresponds to particle approximation error, while the second corresponds to mixing error. The variance increases as α goes to 1.

We show in Appendix ?? that the variance of DMOP-0 is $\tilde{O}(NpG'(\theta)^2/J)$. Previously, Poyiadjis et al. (2011) established that the variance of DMOP-1 is $\tilde{O}(N^4/J)$, ignoring factors of p and G' . Combining these results with Theorem 5 shows that DMOP- α interpolates between the variances of DMOP-0 and DMOP-1, as $N \leq Nk^2 \leq N^4$, with a phase transition as soon as $\alpha < 1$. The phase transition arises because even though the particle filter has forgetting properties (Karjalainen et al., 2023), the resulting derivative estimate of Poyiadjis et al. (2011) does not, and we require both for the variance reduction.

DMOP- α achieves a lower MSE than DMOP-1, as α, k can be as large as desired to balance the impact of the last three terms of Equation 3. We also show that DMOP-0 achieves a MSE of $\tilde{O}(NpG'(\theta)^2(J^{-1} + (1 - \epsilon)^{\lfloor 1/c \log(J) \rfloor}))$, where the second term corresponds to uncontrolled mixing error. Unlike DMOP- α , DMOP-0 has no opportunity to tune α (or k) to reduce said mixing error, leading to uncontrolled asymptotic bias.

8 Computational Efficiency

Our numerical results use an implementation of our methods in the `pypomp` package (Abkemeier et al., 2025). The source code for this paper is at <https://github.com/pypomp/dmop>. DMOP- α and IFAD are fast algorithms, both in theory and practice: the cheap gradient principle of Kakade and Lee (2018) suggests that a gradient estimate from DMOP- α should take no more than 6 times that of the runtime of the particle filter, and we found the empirical value of this factor to be 3.75. DMOP- α and IFAD therefore share the same $O(NJ)$ time complexity as the particle filter, unlike the $O(NJ^2)$ complexity of Corenflos et al. (2021) and Algorithm 2 in (Poyiadjis et al., 2011; Ścibior and Wood, 2021).

Implementation of the particle filter, IF2, and DMOP- α in JAX (Bradbury et al., 2018) takes advantage of just-in-time compilation and GPU acceleration. While CPU-based implementations of the particle filtering methods can effectively leverage multiple cores by parallelizing over independent replications of a given method, `pypomp` can effectively leverage tens of thousands of GPU cores by vectorizing over both particles and independent method replications. To examine the performance improvement on the Dhaka model, we ran 1 instance of IF2 with 60 iterations and 5000 particles using 1 CPU core and compared this with running 100 replications on a GPU. The results in Table 2 show that, despite running 100 times as many replications, the GPU finishes 10.4 times faster, giving it a throughput 1038.9 times greater than the single CPU core.

Metric	<code>pomp</code> (King et al., 2016)	<code>pypomp</code> (Abkemeier et al., 2025)
Language	R (C++ simulator)	Python (JAX)
Hardware	3.0 GHz Intel Xeon Gold 6154 (1 core)	NVIDIA RTX6000 Pro Blackwell
IF2 Replications	1	100
Latency (s)	935	90

Table 2: Comparison of execution latency for running IF2 (60 iterations, 5000 particles, 1 versus 100 replications) between the CPU-only R `pomp` package and the GPU-accelerated `pypomp` package.

9 Discussion

Practical likelihood-based data analysis involves many likelihood optimizations (King et al., 2008; Blake et al., 2014; Pons-Salort and Grassly, 2018; Subramanian et al., 2021; Fox et al., 2022; Drake et al., 2023). This occurs with profile likelihood confidence intervals and model selection. A careful scientist should consider many model variations, to see whether the conclusions of the study are sensitive to alternative model specifications. Coding model variations is relatively simple when using plug-and-play inference methodology. However, assessing the scientific potential of these variations requires likelihood optimization. Monte Carlo adjusted profile methods can help to accommodate maximization and evaluation error (Ionides et al., 2017; Ning et al., 2021) but these methods are most effective when the maximized log-likelihood can be obtained within a few units. Improving the process of model fitting has practical consequences for the ability of scientists to evaluate hypotheses. We have demonstrated, for the first time, a plug-and-play maximum likelihood approach which provides a high-quality log-likelihood maximization in the complex benchmark model of King et al. (2008). We anticipate that this will promote scientific advances in all domains where complex POMP models arise.

If the simulator is discontinuous in θ , DMOP- α no longer applies. We are working on a variation of DMOP- α applicable to this case. Specifically, differentiable transition density ratios can be used instead of a differentiable simulator. The off-parameter treatment of the measurement model is extended to an off-parameter simulation for the dynamic process. In that case, one requires access to the transition densities, or at least their ratios. This algorithm may be better suited to large discrete state spaces than either DMOP- α or differentiation of the Baum-Welch algorithm.

Discounting the weights by some $\alpha \in [0, 1]$ is not the only way to interpolate between the estimators of Naesseth et al. (2018) and Poyiadjis et al. (2011). As the proof of Theorem 5 implies, we can also truncate the weights at a fixed lag, corresponding to the MOP- $(1, k)$ algorithm mentioned. The analysis is similar, with comparable rates but with a slightly less convenient implementation.

Acknowledgments

This research was supported by National Science Foundation grants DMS-1761603 and DEB-1933497. We thank Nicolas Chopin for communications regarding the strong law of large numbers for off-parameter resampled particle filters, and other feedback. We thank Jun Chen, Arnaud Doucet and Kunyang He for helpful discussions regarding the manuscript.

References

- Abkemeier, A. J., Chen, J., Tan, K., Wheeler, J., He, K., Yang, B., King, A. A., Terhorst, J., Hooker, G., and Ionides, E. L. (2025). Pypomp: Inference for partially observed Markov processes with Python and JAX.
- Andrieu, C., Doucet, A., and Holenstein, R. (2010). Particle Markov chain Monte Carlo methods. 72:269–342.
- Arulampalam, M. S., Maskell, S., Gordon, N., and Clapp, T. (2002). A tutorial on particle filters for online nonlinear, non-Gaussian Bayesian tracking. 50:174 – 188.
- Arya, G., Schauer, M., Schäfer, F., and Rackauckas, C. (2022). Automatic differentiation of programs with discrete randomness. *Advances in Neural Information Processing Systems*, 35:10435–10447.
- Bhadra, A., Ionides, E. L., Laneri, K., Pascual, M., Bouma, M., and Dhiman, R. C. (2011). Malaria in Northwest India: Data analysis via partially observed stochastic differential equation models driven by Lévy noise. 106:440–451.
- Blake, I. M., Martin, R., Goel, A., Khetsuriani, N., Everts, J., Wolff, C., Wassilak, S., Aylward, R. B., and Grassly, N. C. (2014). The role of older children and adults in wild poliovirus transmission. 111(29):10604–10609.
- Bradbury, J., Frostig, R., Hawkins, P., Johnson, M. J., Leary, C., Maclaurin, D., Necula, G., Paszke, A., VanderPlas, J., Wanderman-Milne, S., and Zhang, Q. (2018). JAX: composable transformations of Python+NumPy programs.
- Bretó, C. (2014). On idiosyncratic stochasticity of financial leverage effects. 91:20–26.
- Chen, X. and Li, Y. (2024). Normalizing flow-based differentiable particle filters. *IEEE Transactions on Signal Processing*, 73:493–507.
- Corenflos, A., Thornton, J., Deligiannidis, G., and Doucet, A. (2021). Differentiable particle filtering via entropy-regularized optimal transport. In Meila, M. and Zhang, T., editors, *Proceedings of the 38th International Conference on Machine Learning*, volume 139 of *Proceedings of Machine Learning Research*, pages 2100–2111. PMLR.
- Del Moral, P. (2004). *Feynman-Kac Formulae: Genealogical and Interacting Particle Systems with Applications*. Springer, New York.
- Ditlevsen, S. and Samson, A. (2019). Hypoelliptic diffusions: Filtering and inference from complete and partial observations. *Journal of the Royal Statistical Society Series B: Statistical Methodology*, 81(2):361–384.
- Doucet, A., Jacob, P. E., and Rubenthaler, S. (2013). Derivative-free estimation of the score vector and observed information matrix with application to state-space models. *arXiv:1304.5768v3*.
- Drake, J. M., Handel, A., Marty, É., O’Dea, E. B., O’Sullivan, T., Righi, G., and Tredennick, A. T. (2023). A data-driven semi-parametric model of SARS-CoV-2 transmission in the United States. *PLOS Computational Biology*, 19(11):e1011610.

- Fasiolo, M., Pya, N., and Wood, S. N. (2016). A comparison of inferential methods for highly nonlinear state space models in ecology and epidemiology. *31*(1):96–118.
- Fox, S. J., Lachmann, M., Tec, M., Pasco, R., Woody, S., Du, Z., Wang, X., Ingle, T. A., Javan, E., Dahan, M., Gaither, K., Escott, M. E., Adler, S. I., Johnston, S. C., Scott, J. G., and Meyers, L. A. (2022). Real-time pandemic surveillance using hospital admissions and mobility data. *119*(7):e2111870119.
- Griewank, A. and Walther, A. (2008). *Evaluating derivatives: principles and techniques of algorithmic differentiation*. SIAM.
- He, D., Ionides, E. L., and King, A. A. (2010). Plug-and-play inference for disease dynamics: Measles in large and small towns as a case study. *7*:271–283.
- Homan, M. D. and Gelman, A. (2014). The No-U-turn sampler: Adaptively setting path lengths in Hamiltonian Monte Carlo. *J. Mach. Learn. Res.*, *15*(1):1593–1623.
- Ionides, E. L., Bretó, C., and King, A. A. (2006). Inference for nonlinear dynamical systems. *103*:18438–18443.
- Ionides, E. L., Breto, C., Park, J., Smith, R. A., and King, A. A. (2017). Monte Carlo profile confidence intervals for dynamic systems. *14*:1–10.
- Ionides, E. L., Nguyen, D., Atchadé, Y., Stoev, S., and King, A. A. (2015). Inference for dynamic and latent variable models via iterated, perturbed Bayes maps. *112*(3):719–724.
- Jonschkowski, R., Rastogi, D., and Brock, O. (2018). Differentiable particle filters: End-to-end learning with algorithmic priors. *Robotics: Science and Systems XIV*.
- Kakade, S. M. and Lee, J. D. (2018). Provably correct automatic subdifferentiation for qualified programs. *Advances in Neural Information Processing Systems*, 31.
- Karjalainen, J., Lee, A., Singh, S. S., and Vihola, M. (2023). On the forgetting of particle filters. *arXiv:2309.08517*.
- Kim, J. and Stoffer, D. S. (2008). Fitting stochastic volatility models in the presence of irregular sampling via particle methods and the EM algorithm. *Journal of Time Series Analysis*, *29*(5):811–833.
- King, A. A., Ionides, E. L., Pascual, M., and Bouma, M. J. (2008). Inapparent infections and cholera dynamics. *Nature*, *454*:877–880.
- King, A. A., Nguyen, D., and Ionides, E. L. (2016). Statistical inference for partially observed Markov processes via the R package pomp. *Journal of Statistical Software*, *69*:1–43.
- Kingma, D. P. and Ba, J. (2015). Adam: A method for stochastic optimization. In *International Conference on Learning Representations (ICLR)*.
- Knape, J. and de Valpine, P. (2012). Fitting complex population models by combining particle filters with Markov chain Monte Carlo. *Ecology*, *93*(2):256–263.

- Le Cam, L. and Yang, G. L. (2000). *Asymptotics in Statistics*. Springer, New York, 2nd edition.
- Liu, J. and West, M. (2001). Combining parameter and state estimation in simulation-based filtering. In Doucet, A., de Freitas, N., and Gordon, N., editors, *Sequential Monte Carlo Methods in Practice*, pages 197–224. Springer, New York.
- Malik, S. and Pitt, M. K. (2011). Particle filters for continuous likelihood evaluation and maximisation. *Journal of Econometrics*, 165(2):190–209.
- Naesseth, C., Linderman, S., Ranganath, R., and Blei, D. (2018). Variational sequential Monte Carlo. In Storkey, A. and Perez-Cruz, F., editors, *Proceedings of the Twenty-First International Conference on Artificial Intelligence and Statistics*, volume 84 of *Proceedings of Machine Learning Research*, pages 968–977. PMLR.
- Ning, N., Ionides, E. L., and Ritov, Y. (2021). Scalable Monte Carlo inference and rescaled local asymptotic normality. *Bernoulli*, 27:2532–2555.
- Owen, J., Wilkinson, D. J., and Gillespie, C. S. (2015). Likelihood free inference for Markov processes: A comparison. *Statistical Applications in Genetics and Molecular Biology*, 14(2):189–209.
- Pons-Salort, M. and Grassly, N. C. (2018). Serotype-specific immunity explains the incidence of diseases caused by human enteroviruses. *Science*, 361(6404):800–803.
- Poyiadjis, G., Doucet, A., and Singh, S. S. (2011). Particle approximations of the score and observed information matrix in state space models with application to parameter estimation. *Biometrika*, 98(1):65–80.
- Roosta-Khorasani, F. and Mahoney, M. W. (2016). Sub-sampled Newton methods I: Globally convergent algorithms. *arXiv:1601.04737*.
- Rosato, C., Devlin, L., Beraud, V., Horridge, P., Schön, T. B., and Maskell, S. (2022). Efficient learning of the parameters of non-linear models using differentiable resampling in particle filters. *IEEE Transactions on Signal Processing*, 70:3676–3692.
- Ścibior, A. and Wood, F. (2021). Differentiable particle filtering without modifying the forward pass. *arXiv:2106.10314*.
- Singh, A., Makhlouf, O., Igl, M., Messias, J., Doucet, A., and Whiteson, S. (2023). Particle-based score estimation for state space model learning in autonomous driving. *Conference on Robot Learning*, pages 1168–1177.
- Stocks, T., Britton, T., and Höhle, M. (2018). Model selection and parameter estimation for dynamic epidemic models via iterated filtering: Application to rotavirus in Germany. *Biostatistics*, 21(3):400–416.
- Subramanian, R., He, Q., and Pascual, M. (2021). Quantifying asymptomatic infection and transmission of COVID-19 in New York City using observed cases, serology, and testing capacity. *Proceedings of the National Academy of Sciences*, 118(9).

- Svensson, A., Lindsten, F., and Schön, T. B. (2018). Learning nonlinear state-space models using smooth particle-filter-based likelihood approximations. *IFAC-PapersOnLine*, 51(15):652–657.
- Toni, T., Welch, D., Strelkowa, N., Ipsen, A., and Stumpf, M. P. (2009). Approximate Bayesian computation scheme for parameter inference and model selection in dynamical systems. 6:187–202.
- Wood, S. N. (2010). Statistical inference for noisy nonlinear ecological dynamic systems. *Nature*, 466:1102–1104.
- Wycoff, N., Smith, J. W., Booth, A. S., and Gramacy, R. B. (2026). Voronoi candidates for Bayesian optimization. *Journal of Global Optimization*, 94:175–201.